

SICP § 1.3 練習問題 1.29～1.46

問題文・解答・考え方まとめ

プログラミング B 予習・演習用

この資料の範囲

対象範囲は § 1.3 「高階手続きによる抽象の定式化」全体である。

授業計画上の § 1.3.1 「引数としての手続き」、§ 1.3.2 「lambda を使って手続きを構築する」、§ 1.3.3 「汎用手法としての手続き」、§ 1.3.4 「返り値としての手続き」をまとめて扱う。

収録する練習問題は 1.29～1.46 の全 18 問である。問題文は教科書本文をもとに、数式・改行のみ読みやすい形に整えている。

収録問題一覧

問題	該当節	主題	中心となる考え方
1.29	§ 1.3.1 引数としての手続き	シンプソンの公式による数値積分	Simpson 公式、係数つき総和
1.30	§ 1.3.1 引数としての手続き	sum の反復版	反復版 sum、累積変数
1.31	§ 1.3.1 引数としての手続き	product と階乗・ π の近似	product、空の積 1
1.32	§ 1.3.1 引数としての手続き	accumulate への一般化	combiner/null-value による一般化
1.33	§ 1.3.1 引数としての手続き	filtered-accumulate	filter + accumulate
1.34	§ 1.3.2 lambda を使って手続きを構築する	手続きを引数に取る f に f 自身を渡す	手続き適用と型エラー
1.35	§ 1.3.3 汎用手法としての手続き	黄金比を fixed-point で求める	不動点と黄金比
1.36	§ 1.3.3 汎用手法としての手続き	近似値列を表示する fixed-point	不動点列の表示、平均緩和
1.37	§ 1.3.3 汎用手法としての手続き	有限連分数 cont-frac	連分数、再帰版/反復版
1.38	§ 1.3.3 汎用手法としての手続き	Euler の連分数で e を近似する	Euler の連分数
1.39	§ 1.3.3 汎用手法としての手続き	Lambert の連分数で tan を近似する	Lambert の tan 連分数
1.40	§ 1.3.4 返り値としての手続き	三次方程式を作る cubic	手続きを返す cubic
1.41	§ 1.3.4 返り値としての手続き	double の多重適用	double の高階適用
1.42	§ 1.3.4 返り値としての手続き	関数合成 compose	関数合成
1.43	§ 1.3.4 返り値としての手続き	repeated による n 回適用	n 回適用
1.44	§ 1.3.4 返り値としての手続き	smooth と n 重平滑化	smooth の反復
1.45	§ 1.3.4 返り値としての手続き	n 乗根と平均緩和の回数	平均緩和の反復回数
1.46	§ 1.3.4 返り値としての手続き	iterative-improve による抽象化	反復改良法の抽象化

解答コード

```
;; 共通でよく使う基本手続き
(define (square x) (* x x))
(define (cube x) (* x x x))
(define (inc n) (+ n 1))
(define (identity x) x)
(define (average x y) (/ (+ x y) 2))

;; fixed-point 系で使う許容誤差
(define tolerance 0.00001)
```

練習問題 1.29 シンプソンの公式による数値積分

§ 1.3.1 引数としての手続き

問題文（教科書より）

練習問題 1.29: シンプソンの公式は、上で示したものより正確な数値積分の方法である。シンプソンの公式では、関数 f の範囲 a から b の定積分は次のように近似される。

$$\frac{h}{3} (y_0 + 4y_1 + 2y_2 + 4y_3 + 2y_4 + \dots + 2y_{n-2} + 4y_{n-1} + y_n)$$

ここで、 n を適当な偶整数として $h = (b - a)/n$ で、 $y_k = f(a + kh)$ である。 $(n$ を大きくすると近似の精度が上がる)。 f, a, b, n を引数に取り、シンプソンの公式によって計算した定積分の値を返す手続きを定義せよ。その手続きを使い、 $(n = 100$ と $n = 1000$ で) cube の範囲 0 から 1 の定積分を求め、上で示した integral 手続きと結果を比較せよ。

使う考え方・計算式

考え方は、区間 $[a, b]$ を n 等分し、各点 $a + kh$ での値 $f(a + kh)$ に係数を掛けて足すことである。係数は、端点 $k=0, n$ では 1 、奇数番目では 4 、偶数番目では 2 になる。

計算式: $h = (b - a)/n, y_k = f(a + kh)$

近似値: $(h/3) * \sum \text{coefficient}(k) * y_k$

既に出てきた sum を使うと、「 k を 0 から n まで進めて足す」という共通構造を再利用できる。

解答コード

```
(define (square x) (* x x))
(define (cube x) (* x x x))
(define (inc n) (+ n 1))

(define (sum term a next b)
  (if (> a b)
      0
      (+ (term a)
         (sum term (next a) next b))))

(define (simpson f a b n)
  (define h (/ (- b a) n))
  (define (y k)
    (f (+ a (* k h))))
  (define (coefficient k)
    (cond ((or (= k 0) (= k n)) 1)
          ((odd? k) 4)
          (else 2)))
  (define (term k)
    (* (coefficient k) (y k)))
  (* (/ h 3) (sum term 0 inc n)))

(simpson cube 0 1 100)
(simpson cube 0 1 1000)
```

実行結果・確認

$(\text{simpson cube } 0 \ 1 \ 100) \Rightarrow 1/4$ または 0.25

$(\text{simpson cube } 0 \ 1 \ 1000) \Rightarrow 1/4$ または 0.25

cube の 0 から 1 の定積分の正確な値は $1/4$ である。

教科書の integral では $dx=0.01$ で約 0.2499875、 $dx=0.001$ で約 0.249999875 でした。
この例ではシンプソンの公式の方が非常に正確である。

練習問題 1.30 sum の反復版

§ 1.3.1 引数としての手続き

問題文（教科書より）

練習問題 1.30: 上で挙げた sum 手続きは線形再帰を生成する。この手続きは、総和計算が反復的に行われるように書き直すことができる。次の定義の中で欠けている式を埋め、そのやり方を示せ。

```
(define (sum term a next b)
  (define (iter a result)
    (if (??)
        (??)
        (iter (??) (??))))
  (iter (??) (??)))
```

使う考え方・計算式

再帰版では「現在の項 + 残りの総和」という形で、計算の保留が残る。反復版では result に途中までの合計を持たせ、次の a に進むたびに result を更新する。

空の総和は 0 なので、初期値 result は 0 である。

終了条件は a が b を超えたときである。

解答コード

```
(define (sum term a next b)
  (define (iter a result)
    (if (> a b)
        result
        (iter (next a) (+ result (term a)))))
  (iter a 0))

(define (inc n) (+ n 1))
(define (identity x) x)

(sum identity 1 inc 10)
```

実行結果・確認

(sum identity 1 inc 10) => 55

再帰版と計算結果は同じだが、反復版では result が累積値を表すため、計算の保留を増やさずに進められる。

練習問題 1.31 product と階乗・ π の近似

§ 1.3.1 引数としての手続き

問題文（教科書より）

練習問題 1.31:

a. sum 手続きは、高階手続きとして表せる数多くの似たような抽象化の中で、最も単純なものにすぎない。特定範囲の

点における関数の値の積を返す似たような仕組みの手続きを、product という名前で書け。product を使って factorial を定義するやり方を示せ。また、Wallis の公式を使って、product によって π の近似値を計算せよ。

b. あなたが書いた product 手続きが再帰プロセスを生成するのであれば、線形プロセスを生成するものを書け。線形プロセスを生成するのであれば、再帰プロセスを生成するものを書け。

使う考え方・計算式

sum が「足し合わせる」抽象化なら、product は「掛け合わせる」抽象化である。空の和が 0 であるのに対し、空の積は 1 である。

階乗は $1 * 2 * \dots * n$ として product で書ける。

Wallis の公式は次の形にすると product で扱いやすくなる。

$$\pi \approx 2 * \prod_{k=1}^n (4k^2 / (4k^2 - 1))$$

解答コード

```
(define (inc n) (+ n 1))
(define (identity x) x)
(define (square x) (* x x))

;; 再帰版 product
(define (product term a next b)
  (if (> a b)
      1
      (* (term a)
         (product term (next a) next b))))

(define (factorial n)
  (product identity 1 inc n))

(define (pi-approx n)
  (define (wallis-term k)
    (/ (* 4.0 (square k))
       (- (* 4.0 (square k)) 1)))
  (* 2 (product wallis-term 1 inc n)))

;; 反復版 product
(define (product-iter term a next b)
  (define (iter a result)
    (if (> a b)
        result
        (iter (next a) (* result (term a)))))
  (iter a 1))

(factorial 5)
(pi-approx 1000)
```

実行結果・確認

```
(factorial 5) => 120
(pi-approx 1000) => 約 3.1408077460303945
```

n を大きくすると π に近づく。

練習問題 1.32 accumulate への一般化

§ 1.3.1 引数としての手続き

問題文（教科書より）

練習問題 1.32:

a. sum と product (練習問題 1.31) は、どちらも accumulate(集積) というより一般的な概念の特殊なケースであることを示せ。accumulate は、sum や product と同じく項と範囲指定の引数を取り、それに加えて、それまでの項の集積と現在の項をどうやって結合するかを指定する combiner 手続きと、項がなくなったときにどのような基本値を使うかを指定する null-value を引数に取る。accumulate を書き、sum と product の両方を accumulate への単純な呼び出しとして定義できることを示せ。

b. あなたが書いた accumulate が再帰プロセスを生成するのであれば、線形プロセスを生成するものを書け。線形プロセスを生成するのであれば、再帰プロセスを生成するものを書け。

使う考え方・計算式

sum と product の違いは、「結合方法」と「空の場合の値」である。

sum: combiner = +, null-value = 0

product: combiner = *, null-value = 1

したがって、combiner と null-value を引数にすれば、両方を accumulate で表せる。

解答コード

```
(define (inc n) (+ n 1))
(define (identity x) x)

;; 再帰版 accumulate
(define (accumulate combiner null-value term a next b)
  (if (> a b)
      null-value
      (combiner (term a)
                 (accumulate combiner null-value term (next a) next b))))

(define (sum term a next b)
  (accumulate + 0 term a next b))

(define (product term a next b)
  (accumulate * 1 term a next b))

;; 反復版 accumulate
(define (accumulate-iter combiner null-value term a next b)
  (define (iter a result)
    (if (> a b)
        result
        (iter (next a) (combiner result (term a)))))
  (iter a null-value))

(sum identity 1 inc 10)
(product identity 1 inc 5)
```

実行結果・確認

(sum identity 1 inc 10) => 55

(product identity 1 inc 5) => 120

+ と * のような結合を差し替えることで、同じ反復・再帰の枠組みを再利用できる。

練習問題 1.33 filtered-accumulate

§ 1.3.1 引数としての手続き

問題文（教科書より）

練習問題 1.33: 結合する項に対するフィルタ (filter) という概念を導入することで、さらに一般的なバージョンの accumulate (練習問題 1.32) を手に入れることができる。指定された条件を満たす範囲内の値から導出される項だけを結合するというものだ。結果となる filtered-accumulate 抽象は、accumulate と同じ引数に加えて、フィルタを指定する 1 引数の述語を取る。filtered-accumulate を手続きとして書け。次のものを filtered-accumulate を使ってどのように表現するかを示せ。

- a. a から b の区間の素数の二乗の和 (すでに prime? 述語を書いているとする)
- b. n と互いに素である n 未満のすべての正の整数 (つまり、 $\gcd(i,n)=1$ となるすべての整数 $i < n$) の積

使う考え方・計算式

accumulate に「使う項」と「使わない項」を選ぶ predicate を追加する。

predicate(a) が真なら term(a) を combiner で結合する。

predicate(a) が偽なら、その項は無視して次に進む。

これにより、「足す・掛ける」と「条件で選ぶ」を分離できる。

解答コード

```
(define (inc n) (+ n 1))
(define (square x) (* x x))
(define (identity x) x)

(define (filtered-accumulate combiner null-value term a next b predicate)
  (cond ((> a b) null-value)
        ((predicate a)
         (combiner (term a)
                   (filtered-accumulate combiner null-value term (next a) next b predicate)))
        (else
         (filtered-accumulate combiner null-value term (next a) next b predicate))))

;; a. 素数の二乗の和
(define (sum-prime-squares a b)
  (filtered-accumulate + 0 square a inc b prime?))

;; b. n と互いに素な n 未満の正整数の積
(define (relative-prime-product n)
  (define (rel-prime? i)
    (= (gcd i n) 1))
  (filtered-accumulate * 1 identity 1 inc (- n 1) rel-prime?))

;; prime? が定義済みなら、例えば次のように使える
;; (sum-prime-squares 2 10)
;; (relative-prime-product 10)
```

実行結果・確認

(relative-prime-product 10) => 189

10 未満で 10 と互いに素な正整数は 1, 3, 7, 9 なので、 $1 \cdot 3 \cdot 7 \cdot 9 = 189$ である。

prime? が定義済みなら、(sum-prime-squares 2 10) は $2^2 + 3^2 + 5^2 + 7^2 = 87$ になる。

練習問題 1.34 手続きを引数に取る f に f 自身を渡す

§ 1.3.2 lambda を使って手続きを構築する

問題文（教科書より）

練習問題 1.34: 次の手続きを定義するとする。

```
(define (f g) (g 2))
```

すると、次のような結果になる。

```
(f square) => 4  
(f (lambda (z) (* z (+ z 1)))) => 6
```

もしここで、インタプリタに (f f) という組み合わせを評価させるということをする、どのような結果が起こるだろうか。

使う考え方・計算式

f は「引数 g を手続きだと思って、(g 2) を実行する」手続きである。したがって (f f) では、最初は f 自身を g として渡せるが、その次に (f 2) が発生する。

(f 2) は f の中で (2 2) を実行しようとするため、数値 2 を手続きとして適用しようとしてエラーになる。

解答コード

```
(define (f g) (g 2))  
  
(f f)  
  
;; 置換の流れ  
;; (f f)  
;; => (f 2)  
;; => (2 2) ; 2 は手続きではない
```

実行結果・確認

エラーになる。

理由: 2 を手続きとして呼び出そうとするため。

典型的には「application: not a procedure」のようなエラーになる。

練習問題 1.35 黄金比を fixed-point で求める

§ 1.3.3 汎用手法としての手続き

問題文（教科書より）

練習問題 1.35: 黄金比 ϕ (1.2.2 節) が $x \mapsto 1 + 1/x$ という変形の不動点であることを示し、そのことを利用して ϕ を fixed-point 手続きによって求めよ。

使う考え方・計算式

黄金比 ϕ は $\phi^2 = \phi + 1$ を満たす。両辺を ϕ で割ると、 $\phi = 1 + 1/\phi$ である。

つまり、 ϕ は $f(x) = 1 + 1/x$ の不動点である。

不動点とは $f(x) = x$ を満たす x のことなので、fixed-point にこの関数を渡せば ϕ を近似できる。

解答コード

```
(define tolerance 0.00001)

(define (fixed-point f first-guess)
  (define (close-enough? v1 v2)
    (< (abs (- v1 v2)) tolerance))
  (define (try guess)
    (let ((next (f guess)))
      (if (close-enough? guess next)
          next
          (try next))))
  (try first-guess))

(define golden-ratio
  (fixed-point (lambda (x) (+ 1 (/ 1 x))) 1.0))

golden-ratio
```

実行結果・確認

golden-ratio => 約 1.6180327868852458

正確な黄金比は $(1 + \sqrt{5})/2 \approx 1.6180339887$ なので、近い値が得られる。

練習問題 1.36 近似値列を表示する fixed-point

§ 1.3.3 汎用手法としての手続き

問題文（教科書より）

練習問題 1.36: fixed-point を修正し、練習問題 1.22 で示した newline と display という基本手続きを使って、生成する近似値列を表示するようにせよ。次に、 $x \mapsto \log(1000)/\log(x)$ の不動点を求めることによって $x^x = 1000$ の解を求めよ。平均緩和法を使う場合と使わない場合のステップ数を比較せよ。

使う考え方・計算式

$x^x = 1000$ の両辺の自然対数を取ると、 $x \log x = \log 1000$ である。これを x について変形すると $x = \log(1000)/\log(x)$ になる。

よって $f(x) = \log(1000)/\log(x)$ の不動点を求めればよい。

平均緩和法を使う場合は、次の関数の不動点を求める。

$$g(x) = (x + \log(1000)/\log(x)) / 2$$

display を try の中に入れると、各推定値を表示できる。

解答コード

```
(define tolerance 0.00001)

(define (average x y) (/ (+ x y) 2))

(define (fixed-point f first-guess)
  (define (close-enough? v1 v2)
    (< (abs (- v1 v2)) tolerance))
  (define (try guess)
    (newline)
    (display guess)
    (let ((next (f guess)))
      (if (close-enough? guess next)
          next
          (try next))))
  (try first-guess))

;; 平均緩和なし
(fixed-point (lambda (x) (/ (log 1000) (log x)))
  2.0)

;; 平均緩和あり
(fixed-point (lambda (x)
  (average x (/ (log 1000) (log x))))
  2.0)
```

実行結果・確認

平均緩和なし: 約 4.555532270803653 まで収束
平均緩和あり: 約 4.555537551999825 まで収束

目安として、初期値 2.0 から始めると、平均緩和なしでは約 34 回、平均緩和ありでは約 9 回の更新で収束する。

平均緩和により振動が抑えられるため、少ないステップで収束する。

練習問題 1.37 有限連分数 cont-frac

§ 1.3.3 汎用手法としての手続き

問題文（教科書より）

練習問題 1.37:

a. 無限連分数を k 項で打ち切った k 項有限連分数を計算する手続き cont-frac を定義せよ。n と d を 1 引数の手続きとし、それぞれ連分数の項 N_i と D_i を返すとする。(cont-frac n d k) を評価すると k 項有限連分数の値を計算するようにせよ。一連の k の値に対し、 N_i と D_i がすべて 1 の場合によって $1/\phi$ の近似を求め、その手続きをチェックせよ。小数点以下 4 桁の精度の近似を得るには、 k をどれだけ大きくしなければならないだろうか。

b. あなたが書いた cont-frac が再帰プロセスを生成するのであれば、線形プロセスを生成するものを書け。線形プロセスを生成するのであれば、再帰プロセスを生成するものを書け。

使う考え方・計算式

連分数は下から順に計算すると扱いやすい。k 項有限連分数では、最後の項から 0 を初期値として上へ戻っていくと反復版になる。

$N_i = 1, D_i = 1$ の場合、値は $1/\phi \approx 0.6180339887$ に近づく。

解答コード

```
;; 再帰版
(define (cont-frac-rec n d k)
  (define (rec i)
    (if (> i k)
        0
        (/ (n i) (+ (d i) (rec (+ i 1))))))
  (rec 1))

;; 反復版
(define (cont-frac n d k)
  (define (iter i result)
    (if (= i 0)
        result
        (iter (- i 1)
              (/ (n i) (+ (d i) result)))))
  (iter k 0))

(cont-frac (lambda (i) 1.0)
            (lambda (i) 1.0)
            11)
```

実行結果・確認

(cont-frac ... 11) => 約 0.6180555555555556

$1/\phi \approx 0.6180339887$ である。

小数点以下 4 桁の精度を「四捨五入して 0.6180 が得られる」と見れば k=10 程度、誤差を 0.00005 未満に抑えるなら k=11 程度が目安である。

練習問題 1.38 Euler の連分数で e を近似する

§ 1.3.3 汎用手法としての手続き

問題文（教科書より）

練習問題 1.38: スイスの数学者レオンハルト・オイラーは、1737 年に De Fractionibus Continuis という学術報告を発表した。それには、 $e - 2$ の連分数展開が含まれていた。この分数では、 N_i はすべて 1 で、 D_i は 1, 2, 1, 1, 4, 1, 1, 6, 1, 1, ... という数列である。オイラーの連分数展開をもとに、練習問題 1.37 の cont-frac 手続きを使って e を近似するプログラムを書け。

使う考え方・計算式

$e - 2$ を連分数で表せるので、cont-frac の結果に 2 を足せば e の近似値になる。

D_i の並びは、 i が 2,5,8,... のときだけ 2,4,6,... となり、それ以外は 1 である。

つまり $i \bmod 3 = 2$ のとき $D_i = 2 * ((i+1)/3)$ 、それ以外は 1 と書ける。

解答コード

```
(define (cont-frac n d k)
```

```

(define (iter i result)
  (if (= i 0)
      result
      (iter (- i 1)
            (/ (n i) (+ (d i) result))))))

(iter k 0))

(define (e-approx k)
  (+ 2
     (cont-frac (lambda (i) 1.0)
                 (lambda (i)
                   (if (= (remainder i 3) 2)
                       (* 2 (/ (+ i 1) 3))
                       1))
                 k)))

(e-approx 10)
(e-approx 20)

```

実行結果・確認

(e-approx 10) => 約 2.7182817182817183
 (e-approx 20) => 約 2.718281828459045

$e \approx 2.718281828459045$ なので、 $k=20$ ではかなりよく一致する。

練習問題 1.39 Lambert の連分数で $\tan$ を近似する

§ 1.3.3 汎用手法としての手続き

問題文（教科書より）

練習問題 1.39: 正接 (タンジェント) 関数の連分数表現は、1770 年にドイツの数学者ヨハン・ハインリッヒ・ランベルトによって発表された。

$$\tan x = x / (1 - x^2 / (3 - x^2 / (5 - \dots)))$$

ここで、 x はラジアンで表したものである。ランベルトの式をもとに、正接関数の近似値を求める手続き (tan-cf x k) を定義せよ。 k は、練習問題 1.37 同様、計算する項の数を指定する。

使う考え方・計算式

cont-frac は $N_i / (D_i + \text{次の項})$ という形なので、分母の「引き算」は N_i を負にすることで表せる。

Lambert の式では、 $N_1 = x$ 、 $N_i = -x^2$ ($i > 1$)、 $D_i = 2i - 1$ とすればよい。

解答コード

```

(define (cont-frac n d k)
  (define (iter i result)
    (if (= i 0)
        result
        (iter (- i 1)
              (/ (n i) (+ (d i) result))))))
  (iter k 0))

(define (tan-cf x k)
  (cont-frac (lambda (i)
               (if (= i 1)
                   x
                   -x^2))
             (lambda (i)
               (- 1 (+ i 1))))
  k))

```

```
      x
      (- (* x x)))
(lambda (i) (- (* 2 i) 1))
k))
```

```
(tan-cf 1.0 20)
```

実行結果・確認

```
(tan-cf 1.0 20) => 約 1.557407724654902
```

Racket の `(tan 1.0)` も約 1.557407724654902 なので、一致する。

練習問題 1.40 三次方程式を作る cubic

§ 1.3.4 返り値としての手続き

問題文（教科書より）

練習問題 1.40: 次のような形の式で newtons-method 手続きと組み合わせて使うことによって、三次方程式 $x^3 + ax^2 + bx + c = 0$ の零点の近似値を求めることができるような手続き cubic を定義せよ。

```
(newtons-method (cubic a b c) 1)
```

使う考え方・計算式

cubic は a, b, c を受け取り、「x を受け取って $x^3 + ax^2 + bx + c$ を計算する手続き」を返す。

つまり cubic の返り値は数値ではなく、関数である。ここが § 1.3.4 の中心である。

解答コード

```
(define (square x) (* x x))

(define (cubic a b c)
  (lambda (x)
    (+ (* x x x)
      (* a (square x))
      (* b x)
      c)))

;; 例:  $x^3 - 2x - 3 = 0$  を解く関数を作る
((cubic 0 -2 -3) 2)
```

実行結果・確認

```
((cubic 0 -2 -3) 2) => 1
```

これは $2^3 - 2 \cdot 2 - 3 = 1$ だからである。
newtons-method に渡すと、この三次方程式の零点を探せる。

練習問題 1.41 double の多重適用

§ 1.3.4 返り値としての手続き

問題文（教科書より）

練習問題 1.41: 引数がひとつの手続きを引数として取り、その手続きを二回適用する手続きを返す手続き double を定義せよ。例えば、inc が引数に 1 を足す手続きであれば、(double inc) は 2 を足す手続きになる。次の式はどんな値を返すだろうか。

```
((double (double double)) inc) 5)
```

使う考え方・計算式

double は f を受け取り、f を 2 回適用する手続きを返す。

(double inc) は inc を 2 回適用するので +2 である。

(double double) は「手続きを 4 回適用する手続き」を作る効果を持つ。

(double (double double)) まで重ねると、最終的に inc を 16 回適用する。

したがって 5 に 1 を 16 回足し、結果は 21 である。

解答コード

```
(define (inc n) (+ n 1))

(define (double f)
  (lambda (x) (f (f x))))

(((double (double double)) inc) 5)
```

実行結果・確認

```
((double (double double)) inc) 5 => 21
```

inc が 16 回適用されるため、 $5 + 16 = 21$ である。

練習問題 1.42 関数合成 compose

§ 1.3.4 返回值としての手続き

問題文（教科書より）

練習問題 1.42: f と g を二つの 1 引数関数とする。 g に f を合成するということを、関数 $x \mapsto f(g(x))$ として定義する。合成を実装する手続き `compose` を定義せよ。例えば `inc` が引数に 1 を足す手続きであれば、以下ようになる。

```
((compose square inc) 6) => 49
```

使う考え方・計算式

`compose` は f と g を受け取り、「 x を受け取って $f(g(x))$ を計算する手続き」を返す。

返回值が手続きなので、`((compose square inc) 6)` のように、返ってきた手続きをすぐ 6 に適用できる。

解答コード

```
(define (square x) (* x x))
(define (inc n) (+ n 1))

(define (compose f g)
  (lambda (x) (f (g x))))

((compose square inc) 6)
```

実行結果・確認

```
((compose square inc) 6) => 49
```

まず `inc` により 6 が 7 になり、その後 `square` により $7^2 = 49$ になる。

練習問題 1.43 repeated による n 回適用

§ 1.3.4 返回值としての手続き

問題文（教科書より）

練習問題 1.43: f が数値関数で n が正の整数であれば、 f の n 回適用というものを作ることができ、それは x での値が $f(f(\dots$

(f(x))...) である関数として定義できる。入力として f を計算する手続きと正の整数 n を取り、f の n 回適用を計算する手続きを返す手続きを書け。その手続きは、以下のように使えるはずである。

```
((repeated square 2) 5) => 625
```

ヒント: 練習問題 1.42 の compose を使うと便利かもしれない。

使う考え方・計算式

repeated は f を n 回合成した関数を返す。

n=0 のときは何もしない関数 identity とし、n>0 のときは f と repeated の結果を compose する。問題文では n は正の整数だが、n=0 も定義しておくとの後の問題で使いやすくなる。

解答コード

```
(define (square x) (* x x))
(define (identity x) x)

(define (compose f g)
  (lambda (x) (f (g x))))

(define (repeated f n)
  (if (= n 0)
      identity
      (compose f (repeated f (- n 1)))))

((repeated square 2) 5)
```

実行結果・確認

```
((repeated square 2) 5) => 625
```

square を 2 回適用するので、 $5^2 = 25$ 、さらに $25^2 = 625$ である。

練習問題 1.44 smooth と n 重平滑化

§ 1.3.4 返り値としての手続き

問題文（教科書より）

練習問題 1.44: 関数の平滑化 (smoothing) は、信号処理において重要な概念である。f が関数で dx がある小さな値であるとき、f を平滑化したものは x での値が $f(x - dx)$, $f(x)$, $f(x + dx)$ の平均となる関数である。入力として f を計算する手続きを取り、f を平滑化したものを計算する手続きを返す手続き smooth を書け。時には、関数を繰り返し平滑化することによって n 重平滑化関数を得ることが役に立つこともある。smooth と練習問題 1.43 の repeated を使って任意の関数の n 重平滑化関数を生成するやり方を示せ。

使う考え方・計算式

smooth は f を受け取り、新しい関数を返す。その新しい関数は、x の近くの 3 点の平均を返す。

smooth 自体も「関数を受け取って関数を返す関数」なので、repeated smooth n によって smooth を n 回適用できる。

解答コード

```
(define dx 0.00001)
```



```

(define (square x) (* x x))
(define (identity x) x)

(define (compose f g)
  (lambda (x) (f (g x))))

(define (repeated f n)
  (if (= n 0)
      identity
      (compose f (repeated f (- n 1)))))

(define (smooth f)
  (lambda (x)
    (/ (+ (f (- x dx))
          (f x)
          (f (+ x dx)))
       3)))

(define (n-fold-smooth f n)
  ((repeated smooth n) f))

((smooth square) 10)
((n-fold-smooth square 3) 10)

```

実行結果・確認

((smooth square) 10) => 約 100.000000000006667

square はなめらかな関数なので、平滑化しても 10 周辺ではほぼ 100 のままである。

(n-fold-smooth f n) は、smooth を n 回 f に適用した関数を返す。

練習問題 1.45 n 乗根と平均緩和の回数

§ 1.3.4 返り値としての手続き

問題文（教科書より）

練習問題 1.45: 1.3.3 節では、単純に $y \mapsto x/y$ の不動点を探すことによって平方根を計算しようとする収束しないということと、それを平均緩和法によって解決できるということを見た。三乗根の計算にも同じ方法を使うことができ、 $y \mapsto x/y^2$ を平均緩和したもの（不動点として求めることができる）。四乗根では平均緩和を一回するだけでは十分ではなく、平均緩和を二回行くと不動点探索は収束するようになる。 $y \mapsto x/y^{(n-1)}$ を繰り返し平均緩和したものに対する不動点探索によって nth を計算する場合、何回の平均緩和が必要となるか実験せよ。その結果を利用し、fixed-point, average-damp および練習問題 1.43 の repeated 手続きを使って n 乗根を計算する単独の手続きを実装せよ。

使う考え方・計算式

実験すると、必要な平均緩和の回数はおおよそ $\text{floor}(\log_2 n)$ 回になる。

n=2,3 では 1 回

n=4~7 では 2 回

n=8~15 では 3 回

n=16~31 では 4 回

n 乗根は、 $y = x / y^{(n-1)}$ の不動点として表せる。ただしそのままだと発散・振動しやすいので、average-damp を必要回数だけ繰り返してから fixed-point に渡す。

解答コード

```
(define tolerance 0.00001)

(define (average x y) (/ (+ x y) 2))

(define (identity x) x)

(define (compose f g)
  (lambda (x) (f (g x))))

(define (repeated f n)
  (if (= n 0)
      identity
      (compose f (repeated f (- n 1)))))

(define (fixed-point f first-guess)
  (define (close-enough? v1 v2)
    (< (abs (- v1 v2)) tolerance))
  (define (try guess)
    (let ((next (f guess)))
      (if (close-enough? guess next)
          next
          (try next))))
  (try first-guess))

(define (average-damp f)
  (lambda (x) (average x (f x))))

(define (nth-root x n)
  (define damp-times (floor (/ (log n) (log 2))))
  (fixed-point
   ((repeated average-damp damp-times)
    (lambda (y) (/ x (expt y (- n 1)))))
   1.0))

(nth-root 32 5)
(nth-root 65536 16)
```

実行結果・確認

```
(nth-root 32 5)    => 約 2.00000000000021965
(nth-root 65536 16) => 約 2.0000000000076957
```

$32 = 2^5$ 、 $65536 = 2^{16}$ なので、どちらも 2 に近い値が得られる。

練習問題 1.46 iterative-improve による抽象化

§ 1.3.4 返り値としての手続き

問題文（教科書より）

練習問題 1.46: この章で記述した数値計算法のいくつかは、反復改良法 (iterative improvement) と呼ばれる非常に汎用的な計算戦略の実例である。反復改良法というのは、何かを計算するにあたって、答えに対する初期推測値から始めて、その推測値が十分によいかテストし、よくなければ推測値を改良し、改良した推測値を新しい推測値としてそのプロセスを続けるというものである。二つの手続きを引数として取る手続き iterative-improve を書け。引数はそれぞれ、推測値が十分によいか判断する手続きと、推測値を改良する手続きとする。iterative-improve の返り値は、引数として初期推測値を取り、それが十分によくなるまで改良を続ける手続きとする。1.1.7 節の sqrt 手続きと 1.3.3 節の fixed-point 手続きを iterative-improve によって書き直せ。

使う考え方・計算式

sqrt も fixed-point も、共通して次の形である。

1. 推測値を用意する
2. 十分によいか調べる
3. よくなければ改善する
4. 改善後の値で繰り返す

この「反復改良」の骨組みを iterative-improve として抽象化する。iterative-improve は good-enough? と improve を受け取り、初期推測値から反復する手続きを返す。

解答コード

```
(define tolerance 0.00001)

(define (square x) (* x x))
(define (average x y) (/ (+ x y) 2))

(define (iterative-improve good-enough? improve)
  (lambda (first-guess)
    (define (iter guess)
      (if (good-enough? guess)
          guess
          (iter (improve guess))))
    (iter first-guess)))

;; sqrt を iterative-improve で定義
(define (sqrt x)
  ((iterative-improve
    (lambda (guess)
      (< (abs (- (square guess) x)) 0.001))
    (lambda (guess)
      (average guess (/ x guess)))))
  1.0))

;; fixed-point を iterative-improve で定義
(define (fixed-point f first-guess)
  ((iterative-improve
    (lambda (guess)
      (< (abs (- (f guess) guess)) tolerance))
    f)
   first-guess))

(sqrt 9)
(fixed-point cos 1.0)
```

実行結果・確認

```
(sqrt 9) => 約 3.00009155413138
(fixed-point cos 1.0) => 約 0.7390822985224023
```

sqrt と fixed-point は別の問題に見えるが、「十分によいか」と「どう改善するか」を差し替えるだけで同じ反復改良の枠組みとして表せる。